

Custom AI Model — Design, Structure & Roadmap

How the platform lays the groundwork, from day one, for your own model — fed continuously by your data, measured step-by-step against the incumbent AI, and switched over automatically when the data says it's time. Prepared 2026-09-12. Not legal advice; the terms/privacy questions in §7 are flagged for counsel.

1. The framing (important, and precise)

This is **not** a model "trained on Claude," and it is **not** a Claude clone. The design is:

Your first-party data, interpreted/labeled by the incumbent AI, learned by YOUR model.

The incumbent AI (currently Claude, via the existing provider setup) plays two roles only: it **labels/interprets your data** as it operates the platform (its decisions + the human approve/reject on them), and it is the **accuracy benchmark** your model is measured against. The knowledge that ends up in your model comes from **your platform's data and outcomes** — not from copying the incumbent's weights or its general capabilities.

2. The pre-existing structure you plug into (built, from the get-go)

There is one **model interface** the whole platform can call — `serve(domain, context) → { label, confidence }` (`backend/sdk/custom-model.ts`). It routes by an active **backend** setting, so swapping the brain is a config change, never a code rewrite:

- **claude_shadow** (default): the incumbent AI answers for real; **your model runs alongside** as a candidate and its answers are compared to the incumbent's. This is where you live until your model is ready.
- **custom** : **your** model serves. If you've hosted your trained model, set **MODEL_CUSTOM_ENDPOINT** to its URL and the platform POSTs `{domain, context}` to it and uses its answer. If you haven't hosted one yet, the built-in data-learner serves as the custom model. Either way, **money / identity / legal decisions stay human-gated** regardless of which backend answers.

So "plug this data into a pre-existing structure" is literally: the structure is `serve()` + the backend switch; you plug your model in at **MODEL_CUSTOM_ENDPOINT** (or by replacing the built-in predictor) when you have one.

3. Continuous ingestion — fed by every AI process

Everything the platform's AI does already produces the training signal (`backend/sdk/model-training.ts`):

- **Decision labels** — the AI proposed an action in a domain (context) and a human approved/rejected it.
- **Optimization rewards** — a setting change and its measured win/loss/lift.
- **Feature quality** — per-feature success/accuracy/satisfaction.

These are assembled continuously into `(context → label / reward)` examples, **first-party and PII-minimized** (identifiers dropped, free text truncated). As the site runs, the dataset grows on its own — no separate instrumentation.

4. The built-in model (works today, improves as data grows)

Until you host a trained model, the custom backend is a **data-driven learner** over those examples (`predictInternal` — a domain-and-context-conditioned frequency/similarity predictor). It is a real model in the statistical sense: it predicts from your accumulated data and gets better as more arrives. It's the honest placeholder that makes the whole loop live from day one — and it's swappable for a trained neural model without touching anything around it.

5. The switch criterion — EVERY function must EXCEED the existing AI, individually AND as a whole

The site switches to your model exactly when **your model produces more accurate outputs than the pre-existing AI** — and the bar is deliberately strict: it must beat the existing AI **function by function on an individual basis AND as a whole**. `backend/sdk/model-eval.ts` measures both sides against the same **ground truth** (the human's approve/reject on each decision — the final "was this the right call?"):

1. Take the labeled decisions, sort by time; **train** the predictor on the older split, **test** on the newer.
2. On that same held-out test set, for each side: - **Existing-AI accuracy** = how often the pre-existing AI's own proposal was approved (its output was right). - **Your-model accuracy** = how often your model's predicted verdict matches the human's actual verdict.
3. Compute this **overall** and **separately for every function (domain)** — earning, pricing, moderation, advertising, and so on.

The switch gate has **two conditions, and BOTH must be true**:

- **As a whole** — your overall accuracy beats the existing AI's by at least `MODEL_EXCEED_MARGIN_PCT` points (default 0.5, so a statistical tie doesn't flip it), over at least `MODEL_MIN_EVAL_SAMPLES` held-out decisions.
- **Every function individually** — *each* function beats the existing AI by that same margin. A function only counts once it has enough evidence: at least `MODEL_PER_FUNCTION_MIN_SAMPLES` decisions (default 30) to be "confirmed." A function that is **behind** blocks the switch; a function that **doesn't yet have enough data** to confirm *also* blocks it — the model can't switch over on functions it hasn't proven.

An optional absolute floor (`MODEL_ACCURACY_TARGET_PCT` , default 0 = off) can additionally require a minimum accuracy on top of "beats the incumbent."

The model is "ready" — and the site auto-switches — only when every function exceeds the existing AI individually AND the model exceeds it overall. If even one function is behind, or one function still lacks the samples to prove it, the switch stays closed. Both numbers, the margin, the per-function pass/fail breakdown (✓ beating · ✗ behind · 🌀 needs more data), and the trend are on the **Data-Driven Coverage** dashboard, which shows the two gate conditions side by side. Recorded every run (`customModelEval` , scheduled daily) so the gap trends over time.

Honest note on the measurement: because your model only shadows (it isn't making the live decisions yet), its accuracy is scored as how well it predicts the correct verdict on held-out decisions, versus the existing AI's realized hit-rate on the same decisions. It's an apples-to-apples accuracy comparison against the human ground truth; §6's auto-rollback then confirms the win holds up once your model is actually serving.

6. Automatic switchover — on from day one, "when your model is more accurate"

Auto-switch is **ON by default** (`MODEL_AUTO_PROMOTE_ENABLED` , default 1) — from the get-go. When your model has **exceeded** the existing AI — meaning every function individually *and* the whole, per §5 — for `MODEL_READY_STREAK_REQUIRED` consecutive evaluations (default 3, so one lucky run can't trigger it), the active backend **auto-switches to** `custom` and you're notified. (Turn the flag off if you'd rather tap **Promote** yourself on the dashboard; promoting is guarded — it refuses unless your model actually exceeds.)

Auto-rollback (`MODEL_AUTO_ROLLBACK_ENABLED` , default 1) is the safety net: while your model is serving, if it ever stops out-accuracy-ing the existing AI, the backend automatically reverts to shadow and you're notified — so the switch is only ever *kept* while your model is genuinely more accurate. It re-promotes when it pulls ahead again.

Either way the switch changes only *which backend answers* the reversible decisions; the permanent human gates on money / identity / legal are untouched.

7. What to actually train your model with (answering "if not Claude, then what?")

The export (`modelTrainingExport` , gated) is provider-agnostic **JSONL** of (`context → label / reward`) — the standard shape every training path ingests. Your realistic options, from lightest to heaviest:

- **Classical ML on the structured decisions (recommended first).** The approve/reject and optimizer-direction data is tabular — train **XGBoost / LightGBM / scikit-learn** on it. Cheap, fast, runs anywhere, and often matches the incumbent on these structured calls with far less data than a language model needs.
- **Fine-tune an open-source LLM** (you already run Llama via Groq): **Llama 3.x, Mistral, Qwen, or Gemma**, fine-tuned with **LoRA/QLoRA** — on your own hardware or a rented GPU. This becomes a model you fully own.
- **Managed fine-tuning platforms** (upload the JSONL, get a hosted model back): **OpenAI fine-tuning, Google Vertex AI, AWS Bedrock, Together AI, Fireworks AI, Hugging Face AutoTrain, Predibase, Databricks/Mosaic.**

Whichever you pick, you host the result and point `MODEL_CUSTOM_ENDPOINT` at it (or embed a classical model server-side). The harness, ingestion, eval, and switchover don't change.

8. Honest limits

- **Training happens outside this backend.** This Deno service assembles the dataset, serves the built-in learner, measures accuracy, and switches backends — it does **not** train a neural network in-process. Training a net is the external step in §7.
- **The built-in learner is a baseline, not a large model.** It's genuinely data-driven and improves, and it's enough to make the loop and the readiness gate real today; a fine-tuned model in §7 is the upgrade.

9. Terms & privacy — for counsel (do not skip)

- **Provider terms.** Using an AI provider's outputs to help build a separate model can be restricted by that provider's usage terms. Your framing helps (you're training on **your own data with the incumbent's labels**, for **your platform's own operational decisions**, not building a general-purpose competitor) — but whether it is permitted is a **terms-of-service question to confirm against the current provider policy and with counsel**, not something this document decides.
- **Privacy.** Using collected data to train a model is a privacy matter your **privacy policy must disclose** and counsel should clear; the export stays gated (`MODEL_TRAINING_EXPORT_ENABLED` , default OFF, counsel-gated). See `MODEL-TRAINING-DATA-COUNSEL-NOTE.md` .

10. Where it lives (for the reviewer / developer)

- SDK: `backend/sdk/custom-model.ts` (interface, backends, `serve` , built-in learner), `backend/sdk/model-eval.ts` (accuracy-vs-incumbent + auto-promotion), `backend/sdk/model-training.ts` (dataset assembly + readiness).
- Functions: `customModelStatus` , `customModelEval` (scheduled daily), `modelPromote` , `modelReadiness` , `modelTrainingExport` .
- Settings (Automation): `MODEL_BACKEND` , `MODEL_SHADOW_ENABLED` , `MODEL_EXCEED_MARGIN_PCT` (win margin), `MODEL_PER_FUNCTION_MIN_SAMPLES` (decisions needed to confirm a single function, default 30), `MODEL_ACCURACY_TARGET_PCT` (optional absolute floor, 0 = off), `MODEL_MIN_EVAL_SAMPLES` , `MODEL_AUTO_PROMOTE_ENABLED` (default ON — auto-switch), `MODEL_AUTO_ROLLBACK_ENABLED` (default ON — auto-revert on regression), `MODEL_READY_STREAK_REQUIRED` , `MODEL_CUSTOM_ENDPOINT` , `MODEL_TRAINING_ENABLED` , `MODEL_TRAINING_TARGET_EXAMPLES` , `MODEL_TRAINING_EXPORT_ENABLED` .
- Dashboard: **Data-Driven Coverage** (admin) — accuracy vs incumbent, readiness, trend, Promote control.

This is a design/roadmap document, not legal clearance. The provider-terms and privacy questions in §7 and §9 must be reviewed by counsel before enabling the export or switching a live model.